

Validators

1. validators are set of instructions that are used for checking wheher form is working fine or not and data is correct or not

Classification:

there are 2 types of validators

1. User Defined validators or Custom Validators
2. Built in validators

1. Custom Validators:

we create Custom Validators in 2 ways

1. By using normal function
2. By Form class object methods

1. By using normal function:

define a function which is accepting the a value which is responsible for raising the Validation error based on a condition

after defining function connect with field by using validators attribute

Quick Notes Page 24

example:

```
def check_for_a(s):
    if s[0].lower() != 'a':
        raise forms.ValidationError('name should start with a')
```

```
class ContactForm(forms.Form):
    name=forms.CharField(max_length=15,required=True,validators=[check_for_a])
```

2. By Form class object methods

there are 2 Form class object methods

1. clean_element method
we use clean_element method when we have to validate only a single Field(Botcatcher)
2. clean method
we use clean method when we have to validate more than one field

example:

```
class ContactForm(forms.Form):
    botcatcher=forms.CharField(max_length=20,widget=forms.HiddenInput,
                               required=False)
    Email=forms.EmailField(max_length=50)
    Re_Enter_Email=forms.EmailField(max_length=50)
```

```
def clean_botcatcher(self):
    bot=self.cleaned_data.get('botcatcher')
    if len(bot)>0:
        raise forms.ValidationError('data is not entered by human')
```

```
def clean(self):
    e=self.cleaned_data.get('Email')
    r=self.cleaned_data.get('Re_Enter_Email')
```

```
    if e!=r:
        raise forms.ValidationError('emails not matched')
```

HiddenInput widget:

1. we use HiddenInput widget in order to check wheather data submitted is written by Human or any other automated software
2. when we make a field as Hidden-input then that field will not be visible in front form so we can get the form data which is submitted by Human

Note:

It is advised to use clean_element method with only HiddenInput field(botcatcher)

Built_in validators:

in order to make use of this Built_in validators we have to import validators class from django.core

syntax: from django.core import validators

25/43



Built_in validators:

in order to make use of this Built_in validators we have to import validators class from django.core

```
syntax: from django.core import validators
```

There are various kind of validators some of them are:

Quick Notes Page 25

1. RegexValidator
2. MaxValueValidator
3. MinValueValidator
4. MaxLengthValidator
5. MinLengthValidator
6. validate_email
7. validate_url etc

syntax for representation of Built_in validators:

```
from django.core import validators
```

```
class class_name(forms.Form):  
    attribute_name=forms.Field_name(constraints,validators=[validators.validate_email()])
```

example:

forms.py content:

```
class ContactForm(forms.Form):  
    phone=forms.CharField(max_length=10,min_length=10,validators=[validators.RegexValidator('[6-9]\d{9}'))
```

filters

filters are the functionalities which are used for performing some

operations on the data that we r sending through back-end to front-end.
2. we use {} symbol to apply the filters

syntax:

```
{{variable|filter_name:arguments}}
```

Classification of filters:

- Filters are classified into 2 types
1. Built_in filters
 2. User Defined or Custom Filters

1. Built_in filters
these are created by django developers

Some of the Built_in filters are:

1. upper:

it is used to represent the data in UpperCase

example: {{variable|upper}}

2. lower:

it is used to represent the data in LowerCase

example: {{variable|lower}}

3. capfirst:

it is used to represent the first character of entire string on Uppercase and remaining in the case how they r defined.

example: {{variable|capfirst}}

Quick Notes Page 26

4. title:

it is used to represent the first character of each and every word in Uppercase and remaining in lower case

example: {{variable|title}}

26/43



5. length:

it is used to represent the count of the number of characters in the given string

example: `{{variable|length}}`

6. date:

it is used to represent the date in required format

example: `{{variable|date:'dateformat'}}`

Refer below link for date formats:

<https://simpleisbetterthancomplex.com/references/2016/06/21/date-filter.html>

7. truncatewords:

it is used to represent the specified number of words after that (...) will be represented

example: `{{variable|truncatewords:'nummber of words'}}`

8. truncatechars:

it is used to represent the specified number of characters after that (...) will be represented

example: `{{variable|truncatechars:'nummber of characters'}}`

9. slice:

1. it is used to represent the specified number of characters
2. functionality is same as truncatechars but ... will not be represented

example: `{{variable|slice:'nummber of characters'}}`

10. pluralize:

1. it is used to pluralize the given word based on given value
2. other than 1 each and every value is considered as plural value only

example: `{{variable|pluralize:'args'}}`

if u r not passing any args By default pluralize add 's' to the word

1. if u want to add particular pluralization value :

example: `{{variable|pluralize:'es'}}`

2. if u want add either of the value based on count:

example: `{{variable|pluralize:'arg1,arg2'}}`

arg1---->it is displayed for singular values

arg2---->it is displayed for plural values

2. User Defined or Custom Filters

1. in order to create Custom Filters we have to create a templatetags directory inside the application, and create `__init__.py` in order to make it as a package
2. after that create a `__py__` file in which we will write custom Filters

Diagrammatic representation of Application:

```
application/  
  __init__.py  
  models.py  
  templatetags/  
    __init__.py  
    usdfilter.py  
  views.py
```

Content we have to write inside `usdfilter.py` file:

1. import template module from django
`from django import template`
2. Now create a template Library instance and store it in a register variable
`register=template.Library()`
3. Writing custom template filters
Custom filters are Python functions that take one or two arguments

Content we have to write inside usedfilter.py file:

1. import template module from django
from django import template
2. Now create a template Library instance and store it in a register variable
register=template.Library()
3. Writing custom template filters
Custom filters are Python functions that take one or two arguments

arg1---->The value of the variable (input) not necessarily a string.
arg2---->The value of the argument this can have a default value
or be left out altogether.

example of custom filter with both value and arg:

```
def replace_char(value, arg):  
    return value.replace(arg, 'value with we need to replace')
```

example of custom filter with only value:

most of the functions will not be taking arguments so

```
def swap(value): # Only one argument.  
    return value.swapcase()
```

4. Registering custom filters

1. we have to register custom filters with django.template.Library.filter()
2. as we have already created library instance and stored in register variable
this registering of custom filters can be done in 2 ways

1. normal way

The Library.filter() method takes two arguments:

arg1---->The name of the filter a string.
arg2---->The compilation function a Python function
(not the name of the function as a string).

example:

```
register.filter('rep',replace_char)  
register.filter('low',lower)
```

2. decorator method

```
@register.filter(name=nameof filter)  
def functionname(value):  
    return some value
```

5. How to load custom filters into html file

By using load jinja tag we can load custom filters

```
{% load udf_filter_file_name %}
```

Giving customize view to the admin:

1. In order to customize the admin interface we have ModelAdmin class
2. In admin.py we have to create a class which is inherited from ModelAdmin
3. ModelAdmin class is present inside the admin module

Content of admin.py file:

```
from app.models import *  
class WebpageAdminView(admin.ModelAdmin):  
    #values for class members can be provided in the form of list or tuple  
    list_display=['topic_name','name','url']  
    list_editable=('name',)  
    list_per_page=3  
    list_display_links=['url']  
    search_fields=['name','url']  
    list_filter=['url','name','topic_name']
```

```
admin.site.register(Topic)  
admin.site.register(Webpage,WebpageAdminView)  
admin.site.register(AccessRecords)
```